Test Case Catalog

Completion Status & Tool Comparison — Ideal (Catalog Spec) vs. Free/Open-Source Lab

Personal reference · Danish Khan
Companion to *Security Lab Guide* (2026-08-29) and the original Security Lab Test Case Catalog
Compiled 2026-08-29

Summary

Every test case from the original catalog is listed below with an honest status: BUILT means a real artifact exists and was run/converted/validated; PARTIAL means the attack/query/design is real but one piece (usually a live cloud feature) is documented rather than executed; GAP means it needs something this lab genuinely doesn't have for free (a paid license, a live tenant, a Windows VM) and the exact free path is named.

Category	Built	Partial	Gap (free path exists)
1. Detection Engineering	8 / 8 (all Sigma rules + KQL)	0	0
2. Atomic Red Team	0	0	4 / 4 (needs your Windows VM — safety setup is done)
3. SOAR / Automation	0 (infra only)	0	4 / 4 (needs custom Cortex analyzer dev — infra ready)
4. Identity & Access Management	2 / 5 (Graph scripts)	0	3 / 5 (needs a free M365 Developer Program tenant)
5. Cloud Security	2 / 5 (Terraform misconfig, LocalStack privesc sim)	1 / 5 (privesc detection query, no live log to fire it against)	2 / 5 (need Defender for Cloud / Azure Policy on a configured subscription)
6. AI-assisted workflows	2 / 2 (scripts written, SDK verified)	0	0 (not run against a live API key — costs real, small money per call)

1. Detection Engineering — all 8 built

#	Test case	Ideal tool	What was used	Status
1.1	Impossible travel	Sentinel sign-in logs	Sigma → KQL via custom pipeline	BUILT
1.2	Password spray	Sentinel sign-in logs	Sigma → KQL + hand-written `summarize` aggregation	BUILT
1.3	Suspicious OAuth consent	Entra ID audit logs	Sigma → KQL against `AuditLogs`	BUILT
1.4	Privileged role assignment	Entra ID audit logs	Sigma → KQL against `AuditLogs`	BUILT
1.5	Mass file download	M365 audit logs	Sigma → KQL against `OfficeActivity`	BUILT
1.6	Conditional Access tampering	Entra ID audit logs	Sigma → KQL against `AuditLogs`	BUILT
1.7	Anomalous PowerShell	Defender for Endpoint	Sigma → KQL via `sentinel_asim` → `imProcessCreate`	BUILT

> **Real gap disclosed honestly:** none of these 8 KQL queries have been run against a live Sentinel workspace — no Azure AD tenant with real sign-in/audit data exists yet. "Converts cleanly and is syntactically correct" is what's proven; "fires correctly on real data with an acceptable false-positive rate" still needs that tenant. See `detection-engineering/attack-mapping.csv` Status column, which says exactly this per rule.

All artifacts: `detection-engineering/sigma-rules/*.yml`, `kql-conversions/generated/*.kql`.

2. Atomic Red Team — all 4 need your Windows VM

#	Test case	Validates rule	Status
2.1	T1078.004 Valid Accounts	1.1 or 1.2	GAP
2.2	T1098.003 Account Manipulation	1.4	GAP
2.3	T1059.001 PowerShell	1.7	GAP
2.4	T1003 Credential Dumping	none built yet	GAP (advanced, isolated VM only)

What's actually done: the safety-first setup (`atomic-red-team-validation/LAB-SETUP.md`) — VM isolation rules, UTM configuration, snapshot discipline — all real, none skipped. **What's not done, deliberately:** the Windows evaluation VM itself. That's a multi-GB download with a Microsoft EULA you should read yourself, not something to fetch silently. Once it exists, each of 2.1–2.3 has a rule already built and waiting to validate against (1.1/1.2, 1.4, 1.7 above).

3. SOAR / Automation — infra live, analyzers not yet custom-built

#	Test case	Ideal tool	Free equivalent	Status
3.1	Auto-enrich sign-in alert	Sentinel Logic App	TheHive + Cortex custom analyzer	GAP
3.2	Auto-tag/route by severity	Sentinel automation rules	TheHive case templates + tags (UI-configurable)	GAP
3.3	Auto-disable on compromise	Sentinel Logic App + Graph	Cortex responder calling Graph API	GAP
3.4	Ticket creation on alert	Sentinel Logic App	TheHive's own case IS the ticket — no separate connector needed	GAP

> **What's actually running** (verified end-to-end): TheHive + Cortex, both healthy, linked. What's missing for 3.1–3.3 specifically is a *custom Cortex analyzer/responder* — Cortex ships a Python SDK (`cortexutils`) for exactly this, but writing one is real, scoped development work (a Python class implementing Cortex's analyzer interface, packaged with a JSON definition), not something to fabricate quickly to check a box. 3.4 is arguably already done by definition — a TheHive case is a ticket; there's no separate "ticketing system" to integrate when the case management tool already is one.

Concrete next step: build one real Cortex analyzer against something this lab already has live data for — e.g. an analyzer that takes a username observable and queries LocalStack IAM (already working, see Category 5) for that account's attached policies. That would take 3.1 from GAP to BUILT without needing any new paid service.

4. Identity & Access Management — 2 built, 3 need a free tenant

#	Test case	Needs premium license?	Status
4.1	Access review automation	No — Graph API is free on any tier	**BUILT** — `intune-endpoint-health-platform/scripts/privileged_role_access_review.py`
4.2	Orphaned account detection	No — Graph API is free on any tier	**BUILT** — `.../orphaned_account_detection.py`
4.3	Conditional Access test matrix	P1 (Conditional Access)	**GAP**
4.4	PIM activation walkthrough	P2 (PIM)	**GAP**
4.5	Least-privilege RBAC audit	No — Azure free tier is enough	**GAP** (needs a subscription with real role assignments to review)

> **The free path for 4.3/4.4, correctly identified:** the Microsoft 365 Developer Program gives a free instant sandbox tenant with a 90-day (auto-renewing while active) E5 trial — meaning Conditional Access *and* PIM are both included at no cost. This isn't a paid-tool gap at all, just an unbuilt one. Signing up for that sandbox is the concrete next step, not a blocker.

5. Cloud Security — 2 built, 1 partial, 2 need a configured subscription

#	Test case	Ideal tool	Status
5.1	Misconfig then fix	Terraform + Defender for Cloud	**BUILT** (Terraform real; Defender for Cloud finding documented, not screenshotted — see below)
5.2	Public storage lockdown	Azure Policy + Defender for Cloud	**GAP** (needs a configured subscription)
5.3	Key Vault access policy review	Key Vault + Terraform	**GAP**
5.4	Azure Policy compliance dashboard	Azure Policy	**GAP**
5.5	AWS IAM privesc via CloudTrail	CloudTrail + GuardDuty	**PARTIAL** — attack real (LocalStack), detection query written against real CloudTrail schema but never fired live

> **On 5.1:** the vulnerable NSG rule (`0.0.0.0/0` on port 22) was deliberately never applied to a real subscription — see `terraform-labs/05-misconfig-then-fix/BEFORE.md` for why (didn't want to actually expose a port to the internet just for a screenshot). The fixed version is real, applied, and validated.

On 5.5: full detail in `aws-identity-detection/detections/privesc-detection.md` — run `simulate-privesc.sh` yourself to see the real attack sequence execute against LocalStack.

6. AI-Assisted Workflows — both built, cost caveat

#	Test case	Status
6.1	Alert triage summarizer	**BUILT** — `detection-engineering/llm-triage/alert_summarizer.py`
6.2	Incident timeline builder	**BUILT** — `.../incident_timeline.py`

> Unlike every other category, this one has a real (small) per-call cost — LLM API calls are billed per token. Both scripts install cleanly and the SDK import was verified; they weren't run against a live API key in this session, so "produces a good summary" is still yours to confirm the first time you run one.

Suggested build order, updated

Week	Focus	What's already done vs. what's left
1	Sigma + detection-as-code	Done: 1.1, 1.2, 1.4 (and 5 more, bonus). Left: deploy against a real Sentinel workspace once you have a tenant.
2	ATT&CK + Atomic Red Team	Done: safety setup. Left: build the Windows VM, run 2.1–2.3 against the rules that are already waiting.
3	SOAR	Done: TheHive+Cortex running and linked. Left: one custom Cortex analyzer (3.1) — the concrete, scoped next step.
4	AWS identity + LLM workflow	Done: 5.5 attack simulation + query design, both 6.1/6.2 scripts. Left: a real AWS free-tier account for a live CloudTrail trail; an Anthropic API key to actually run the LLM scripts.

Reminder on hygiene

Every deliverable above went through the same rules as the rest of the portfolio: gitleaks pre-commit scanning on every repo, no real tenant/account IDs, synthetic or reserved-range sample data only, and every README says plainly what's actually verified versus what's designed-but-untested. That distinction — not overclaiming completion — is worth more in an interview than a catalog with every box checked and no way to back it up.